flutter_invoice_generator

Learning Guide - how the package is designed and built, phase by phase, in simple language

Chapter 1: Phase 1 - Architecture Plan
Chapter 2: Phase 2 - Package Skeleton and Models
Chapter 3: Phase 3 - Calculation Engine
Status: Phase 3 complete.

1. What are we building?

We are building a **Flutter package** (also called a library or SDK). A package is not an app. Nobody 'opens' it on their phone. Instead, other Flutter developers add it to their own apps with one line in `pubspec.yaml`, and then they get ready-made classes and functions they can call.

Our package gives them one job, done very well: **turn invoice data into a professional PDF**, then let them preview, save, share or print it.

> **In simple words**
> Think of the package as a machine. Developers put in data (who sells, who buys, what items, which tax) and the machine gives back a finished PDF invoice. They never need to know how the machine works inside.

The developer experience we want

```
final invoice = Invoice(
  invoiceNumber: 'INV-1001',
  issueDate: DateTime.now(),
  dueDate: DateTime.now().add(const Duration(days: 30)),
  seller: Business(name: 'ABC Technologies', email: 'hello@abc.com'),
  customer: Customer(name: 'John Doe', email: 'john@example.com'),
  items: [
    InvoiceItem(name: 'Flutter App Development', quantity: 1, unitPrice: 500),
    InvoiceItem(name: 'UI/UX Design', quantity: 2, unitPrice: 100),
  ],
  tax: Tax(name: 'VAT', percentage: 13),
  discount: Discount.percentage(10),
  currency: Currency.usd,
);

final pdf = await InvoiceGenerator.generate(invoice);  // build the PDF
await InvoiceGenerator.share(invoice);                 // share sheet
await InvoiceGenerator.print(invoice);                 // print dialog
await InvoiceGenerator.save(invoice);                  // save to a file
```

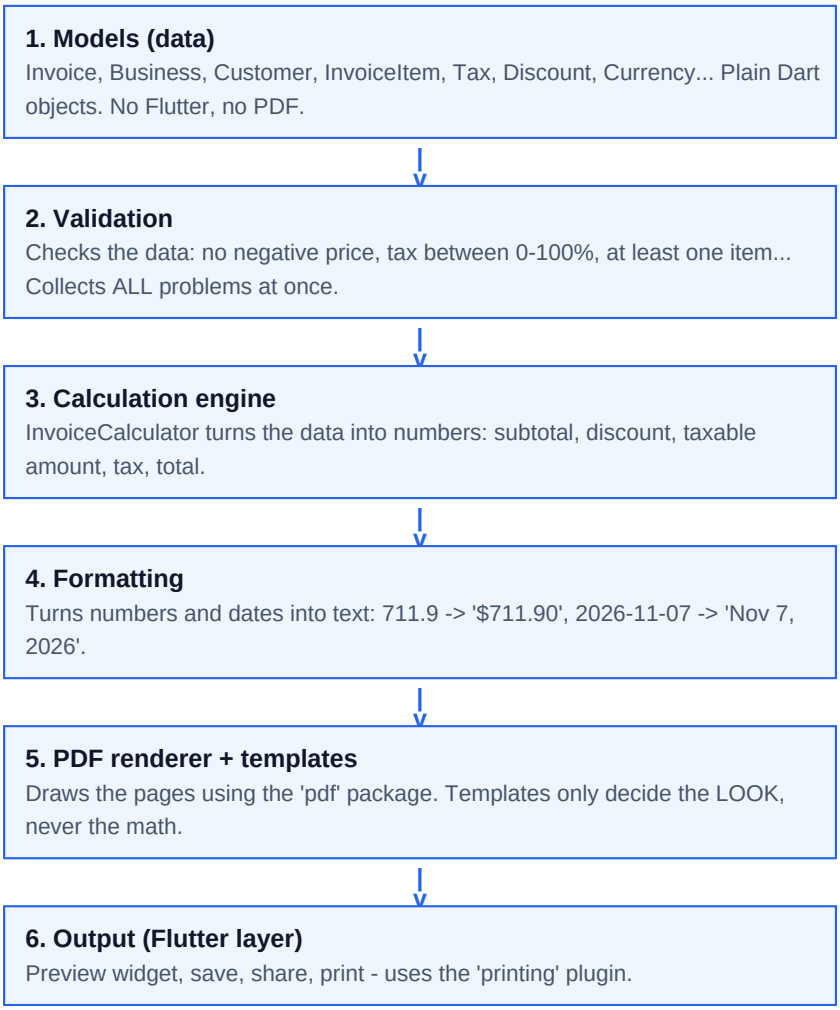
Important: your current folder

Right now `pdf_invoice/` was created with `flutter create` as an **app**. It has `android/ ios/ web/ macos/ windows/ linux/` folders and a `main.dart`. A package does not need those at the root. In Phase 2 we convert it:

- Remove the platform folders and `lib/main.dart` from the root (they are only generated boilerplate, no code of yours).
- Rename the package to `flutter_invoice_generator` in `pubspec.yaml`.
- Create a separate demo app inside `example/`. That demo app is where the platform folders belong.

2. The big picture: layers

The most important idea in this package is **separation**. Each layer does one job and only talks to the layer next to it. This keeps the code easy to test and easy to change later.



> **In simple words**
>
> It is like a restaurant. Models are the ingredients, validation is checking the ingredients are fresh, the calculator is the recipe, templates are how the plate is decorated, and output is the waiter delivering the plate. The decorator never changes the recipe.

Why this matters

- **Testing:** we can test the money math with no PDF at all - fast and exact.
- **Templates:** Classic, Modern and Minimal share the same engine. Only the drawing code differs, so a bug fixed in the math is fixed for all.
- **Server use:** layers 1-5 do not need Flutter UI, so the same code could later generate invoices on a Dart server.

3. Folder structure

This follows the structure you asked for, with a few small additions (marked NEW) that I explain below.

```
flutter_invoice_generator/
|-- lib/
|   |-- flutter_invoice_generator.dart   <- the ONLY public entry file
|   `-- src/                             <- private implementation
|       |-- models/
|       |   |-- invoice.dart
|       |   |-- invoice_item.dart
|       |   |-- business.dart
|       |   |-- customer.dart
|       |   |-- tax.dart
|       |   |-- discount.dart
|       |   |-- currency.dart
|       |   |-- payment_information.dart
|       |   `-- invoice_logo.dart          NEW: bytes / asset / network logo
|       |-- calculation/
|       |   |-- invoice_calculator.dart
|       |   |-- invoice_totals.dart
|       |   `-- money.dart                 NEW (internal): exact money math
|       |-- validation/                    NEW
|       |   `-- invoice_validator.dart
|       |-- formatting/                    NEW
|       |   `-- invoice_formatter.dart     dates + money text
|       |-- pdf/
|       |   |-- invoice_generator.dart     public facade (generate/save/...)
|       |   |-- invoice_pdf.dart           the result object
|       |   |-- invoice_pdf_builder.dart   internal engine (MultiPage)
|       |   |-- invoice_theme.dart         colors, fonts, sizes
|       |   |-- invoice_labels.dart        NEW: 'INVOICE', 'Bill To'... (translatable)
|       |   |-- sections/                  NEW: shared building blocks
|       |   |   (header, parties, items_table, summary, footer)
|       |   `-- templates/
|       |       |-- invoice_template.dart
|       |       |-- classic_template.dart
|       |       |-- modern_template.dart
|       |       `-- minimal_template.dart
|       |-- preview/
|       |   `-- invoice_preview.dart
|       |-- output/                        NEW: save / share / print
|       |   `-- invoice_output.dart
|       `-- exceptions/
|           `-- invoice_exceptions.dart
|-- assets/fonts/                         NEW: bundled Unicode font
|-- example/                             demo Flutter app
|-- test/  (models/ calculation/ validation/ pdf/)
|-- docs/learning/                       this guide (not published)
|-- README.md  CHANGELOG.md  LICENSE
|-- pubspec.yaml  analysis_options.yaml
```

Why lib/src?

In Dart, anything under `lib/src/` is considered private. Developers only import `package:flutter_invoice_generator/flutter_invoice_generator.dart`, and that file decides exactly what is visible using `export`. If a class is not exported, we can rename or rewrite it later without breaking anyone's app.

In simple words

lib/src is the kitchen. The export file is the menu. Customers only order from the menu, so we can reorganise the kitchen any time.

4. Dependencies (and why each one)

I checked pub.dev on 7 Nov 2026 for the current versions. Fewer dependencies means fewer version conflicts for the people who install us.

Package	Version	Why we need it	Could we avoid it?
pdf	^3.13.1	Builds the PDF: pages, tables, text, images, MultiPage for long invoices. Also includes 'barcode', so QR codes are free.	No - this is the core.
printing	^5.15.1	Print dialog, share sheet, PDF preview widget, loading network images. Supports Android, iOS, macOS, Windows, Linux, Web.	No - writing native print code for 6 platforms is huge.
intl	>=0.19.0 <0.21.0	Date formats ('07/11/2026') and number grouping ('1,234.50') for any locale.	Possible, but locale rules are hard. A WIDE range is used because Flutter's own localization pins one intl version.
path_provider	^2.x (to confirm)	Only for save(): finds the app documents folder on each platform.	Yes - see decision below.
meta	(already via Flutter)	@immutable annotations.	Free.

Decision for you to approve

save(): Option A (recommended): use path_provider so `save(invoice)` works with no arguments and returns the saved file path. Option B: no extra dependency, but the developer must pass a directory: `save(invoice, directory: dir)`. On Web, save() downloads the file through the browser either way.

A hidden problem: currency symbols and fonts

PDFs need a font file that contains every character we print. The built-in PDF font (Helvetica) does **not** contain the Indian Rupee sign or many non-English letters. A Nepali shop printing a customer name in Devanagari would get empty boxes.

Decision for you to approve

Bundle one open-licence font (Noto Sans, regular + bold, about 1 MB) as the default, which includes symbols like the Rupee sign. Developers can still pass any font through InvoiceTheme. This keeps generation **fully offline** (no downloading Google Fonts at runtime).

5. The models (the data)

All models are **immutable**: once created they never change. To 'edit', you call `copyWith(...)` which returns a new copy. This prevents bugs where one screen changes an invoice that another screen is still printing.

```
Invoice
 |-- invoiceNumber, issueDate, dueDate
 |-- seller   : Business   (name, email, phone, address, website,
 |                          taxId, registrationNumber, logo)
 |-- customer : Customer   (name, companyName, email, phone, address, taxId)
 |-- items    : List<InvoiceItem>
 |                (name, description, quantity (double), unitPrice,
 |                 sku, unit, tax?, discount?)
 |-- currency : Currency    (code, symbol, decimalDigits, symbolOnLeft)
 |-- taxes    : List<Tax>   (name, percentage)  <- 'tax:' shortcut for one
 |-- discount : Discount?   (percentage or fixed)
 |-- paymentInformation : PaymentInformation? (all fields optional)
 |-- notes, terms
 |-- details  : List<InvoiceDetail>  (extra rows shown, e.g. 'PO Number')
 `-- metadata : Map<String, String>  (your own data, never printed)
```

Design choices explained

- **Quantity is a double**, so 2.5 hours or 0.75 kg work.
- **Multiple taxes:** you can write `tax: Tax(...)` for the simple case or `taxes: [vat, serviceTax]`. No country rules are hard-coded - we only use the percentage you give.
- **Discount** gets friendly constructors: `Discount.percentage(10)` and `Discount.fixed(50)`. The long form `Discount(type: DiscountType.percentage, value: 10)` also works.
- **Currency** has ready constants (usd, eur, gbp, inr, npr, aud, cad, jpy) and you can make your own: `Currency(code: 'NPR', symbol: 'Rs.', decimalDigits: 2)`. JPY uses 0 decimals.
- **Logo** can come from bytes, an asset or (only if you ask) the network.
- **Optional fields everywhere** - adding a new optional field in v1.1 does not break anyone. That is how we stay backward compatible.

> **Decision for you to approve**
>
> You listed 'template configuration' as an Invoice field. I recommend **not** putting it inside Invoice. An invoice is data (what was sold); the template is presentation (how it looks). Instead: `InvoiceGenerator.generate(invoice, template: InvoiceTemplate.modern, theme: myTheme)`. Then the same invoice can be printed in any style, and saving invoices to a database never stores colours.

6. The calculation engine

Money math is where invoice software most often goes wrong. Computers store decimals like 0.1 in binary and cannot store them exactly:

```
print(0.1 + 0.2);      // 0.30000000000000004   (not 0.3!)
print(1.005 * 100);    // 100.49999999999999    (should be 100.5)
```

Our strategy: count in the smallest unit

Internally we never add doubles. We convert every amount into **whole minor units** (cents for USD, paisa for NPR, yen for JPY) stored as exact integers, do the math, and convert back only for display. We read the number through its shortest text form (1.005 stays '1.005'), so the binary error never enters the calculation. This needs **no extra package**.

> **In simple words**
> Instead of saying 'one dollar and five cents' (1.05) we say '105 cents'. Whole numbers never have rounding surprises.

The formula (worked example)

Step	Rule	Example
Line total	quantity x unitPrice - item discount	1 x 500 = 500.00 ; 2 x 100 = 200.00
Subtotal	sum of line totals	700.00
Discount	invoice discount on the subtotal	10% of 700 = 70.00
Taxable amount	subtotal - discount	630.00
Tax	each tax % of the taxable amount	VAT 13% of 630 = 81.90
Total	taxable amount + all taxes	711.90

Rounding rules (documented in code and README)

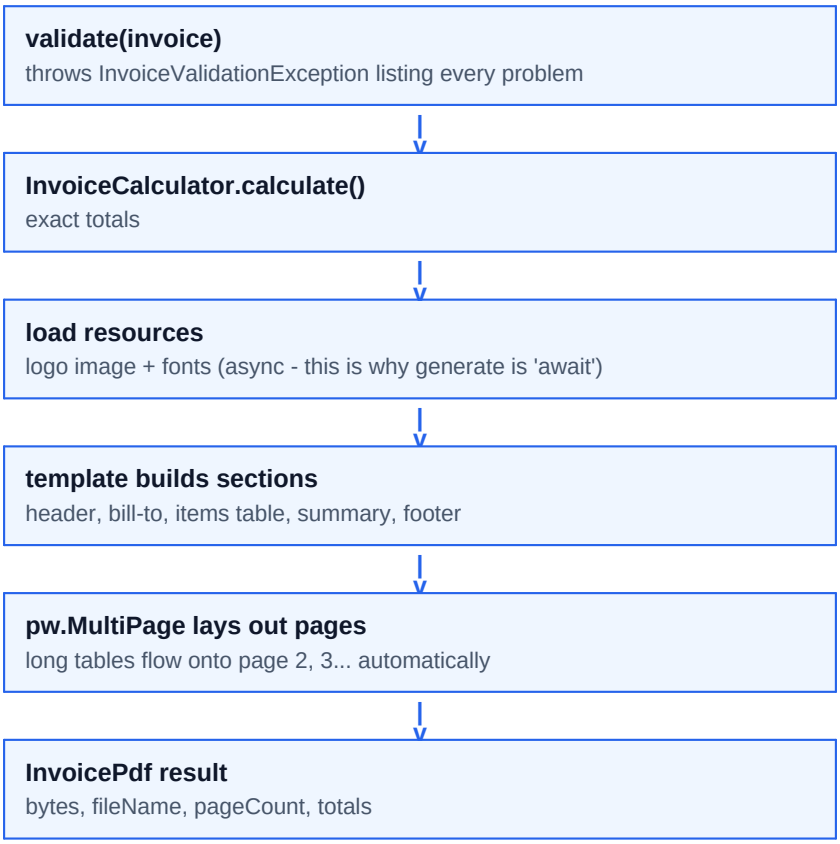
- Rounding happens to the currency's decimal places (2 for USD, 0 for JPY).
- Mode: **round half away from zero** (0.125 -> 0.13), the normal commercial rule.
- We round each line total, the discount and each tax amount once. The total is then an exact sum, so the numbers on the PDF always add up.
- If an item has its own tax, that item is taxed with its own rate; the invoice discount is shared across items in proportion to their value (leftover cents go to the lines with the largest remainder - see Chapter 3).

The result is an immutable `InvoiceTotals` object: `subtotal, discountAmount, taxableAmount, taxLines (one per tax), taxAmount, total, lines`. Developers can use it without making a PDF at all:

```
final totals = InvoiceCalculator.calculate(invoice);
print(totals.total); // 711.9
```

7. PDF generation and templates

What generate() does



What you get back

```
final pdf = await InvoiceGenerator.generate(invoice);
pdf.bytes;      // Uint8List - upload it, email it, store it
pdf.fileName;   // 'INV-1001.pdf'
pdf.pageCount;  // 3
pdf.totals;     // the same numbers printed on the PDF
```

Returning our own small `InvoicePdf` class (instead of raw bytes) means we can add fields later without breaking anyone.

Multiple pages

The pdf package has a widget called `MultiPage`. We give it a long list of content; it fills page 1, then continues on page 2. The items table header row repeats on each new page, the big business header is drawn only on page 1, and every page gets 'Page 2 of 3' in the footer. We will test 1, 10, 50 and 150 items and check the real page count.

Templates without copy-paste

Every invoice has the same five **sections**. Shared section builders live in `pdf/sections/`. A template is a small class that decides how each section looks (colours, alignment, header band or not), reusing those builders.

Template	Look
Classic	Traditional: logo left, 'INVOICE' right, bordered table, grey header row.
Modern	SaaS style: coloured header band, rounded summary card, zebra rows.
Minimal	Lots of white space, thin lines, no fills, small type.

> **Decision for you to approve**
>
> In v1, `InvoiceTemplate` exposes only the three built-in templates (`InvoiceTemplate.classic`, etc.). Opening it up for fully custom templates is planned, but doing it in v1 would freeze our internal drawing code as public API. Keeping it closed now lets us improve it freely.

Customization: InvoiceTheme and InvoiceLabels

```
InvoiceTheme(
  primaryColor: PdfColor.fromHex('#2563EB'),
  secondaryColor: PdfColors.grey700,
  font: myFont, boldFont: myBoldFont,
  baseFontSize: 10, spacing: 12, borderRadius: 6,
  pageFormat: PdfPageFormat.a4,           // or PdfPageFormat.letter
  dateFormat: 'dd/MM/yyyy', locale: 'en_US',
)
InvoiceLabels(invoiceTitle: 'TAX INVOICE', billTo: 'Bill To', ...)
```

Labels are separate so a Nepali or French developer can translate every word on the PDF without touching colours.

8. Preview, save, share, print

Preview widget

`InvoicePreview(invoice: invoice)` is a Flutter widget. Two ways to build it:

Option	Good	Bad
A. Show the real PDF (recommended)	Looks exactly like the PDF, always. Little code. Uses printing's PdfPreview.	Slightly slower to appear.
B. Re-draw it with Flutter widgets	Instant, scrollable.	Two layouts to keep in sync - they WILL drift apart over time.

Save / share / print

Platform	Print	Share	Save
Android	yes	yes (share sheet)	yes (app documents)
iOS	yes	yes (share sheet)	yes (app documents)
macOS	yes	yes	yes (needs sandbox file permission)
Windows	yes	save dialog	yes
Linux	yes	save dialog	yes
Web	browser print	browser download	browser download

This table is from the printing package docs. In Phase 7 I will verify what I can on your Mac and clearly mark anything not tested.

Error handling

```
InvoiceException                    (base - catch everything)
 |-- InvoiceValidationException     bad data. Has .issues: a list like
 |                                  ['items[2].quantity must be >= 0',
 |                                   'tax VAT: percentage must be 0-100']
 |-- InvoiceImageException          logo missing / broken / unsupported
 `-- InvoiceGenerationException     anything else while building the PDF
```

Validation collects **all** problems at once, so a developer fixes them in one go instead of one crash at a time. Rules: invoice number not empty, at least one item, quantity >= 0, price >= 0, tax 0-100%, percentage discount 0-100%, fixed discount not bigger than the subtotal, due date not before issue date.

QR codes

Not mandatory in v1. Because 'pdf' already includes the barcode library, an optional `InvoiceQrCode` (payment URL, invoice URL or custom text) costs zero new dependencies. I reserve an optional field for it and plan it for Phase 5 as a small extra.

9. The plan, phase by phase

Phase	What	You push after?
1	Architecture plan + this guide (now)	Yes - first commit
2	Convert to package, models, exports, model tests	Yes
3	Calculation engine + many tests	Yes
4	PDF generator + Classic template + multi-page tests	Yes
5	Theme, labels, Modern + Minimal templates, (QR)	Yes
6	InvoicePreview widget	Yes
7	save / share / print + platform table	Yes
8	Example app	Yes
9	README, CHANGELOG, API docs	Yes
10	pub.dev dry-run, fix everything	Yes - then publish

After every phase I stop, tell you what I did and how, add a chapter to this guide, and give you a commit message. I never commit or push - you do.

How to push (first time)

```
cd /Users/prajan/Desktop/package/pdf_invoice
git init
git add .
git commit -m "docs: add architecture plan and learning guide (phase 1)"
git branch -M main
git remote add origin https://github.com/<you>/flutter_invoice_generator.git
git push -u origin main
```

Future pro features (not in the free core)

Extra premium templates, advanced branding, custom sections, recurring invoices. The core stays a clean, free SDK - no payments or licences inside the package.

Chapter 2 - Phase 2: Skeleton and models

In this phase we turned the folder into a real package, wrote every data model, the error classes, the public export file, and 42 tests. There is no PDF code yet - that is on purpose. We first make the data solid, then build on top of it.

Step 1 - From app to package

What we did	Why
Deleted android/ ios/ web/ macos/ windows/ linux/, lib/main.dart, test/widget_test.dart	A package has no app to run. Those folders were the default counter app. The demo app will live in example/ (Phase 8).
Changed .metadata to `project_type: package`	Tells the Flutter tool this is a package, not an app.
Renamed to `flutter_invoice_generator` in pubspec.yaml	The name people will type in their pubspec. Users import `package:flutter_invoice_generator/...`.
Kept `publish_to: 'none'` for now	A safety lock so nothing gets published by accident. Removed in Phase 10.
Only one dependency added: `meta`	For `@immutable`. Flutter already ships it, so users download nothing new. pdf/printing come in Phase 4.
Stricter analysis_options.yaml	Forces doc comments on every public API, trailing commas, final locals, strict types. pub.dev scores packages on this.

Step 2 - The models

```
lib/src/models/
  currency.dart            Currency + 8 built-ins (usd, eur, gbp, inr,
                           npr, aud, cad, jpy) and Currency.values
  tax.dart                 Tax(name, percentage)
  discount.dart            Discount, DiscountType, .percentage(), .fixed()
  business.dart            seller (name required, the rest optional)
  customer.dart            buyer  (name required, the rest optional)
  payment_information.dart all optional + isEmpty
  invoice_logo.dart        InvoiceLogo.memory / .asset / .network
  invoice_detail.dart      extra header rows like 'PO Number'
  invoice_item.dart        one line
  invoice.dart             everything together
```

Five Dart ideas used everywhere (learn these!)

1. Immutable + final fields. Every field is `final` and the class is marked `@immutable`. Once made, an invoice never changes.

2. Defensive copies. If you pass a list to an Invoice and later change your list, the invoice must not change. So the constructor copies it:

```
Invoice({required List<InvoiceItem> items, ...})
    : items = List.unmodifiable(items);   // copy + read-only
```

3. copyWith. To 'edit' an immutable object you make a changed copy:

```
final updated = invoice.copyWith(invoiceNumber: 'INV-1002');
```

Note: with this common pattern, passing `null` means 'keep the old value', so copyWith cannot remove an optional field. We documented that.

4. Value equality (== and hashCode). By default Dart says two objects are equal only if they are the *same object*. We override == so two invoices with the same data are equal. The rule: if you override ==, you must override hashCode the same way, or Sets and Maps break.

5. final class. `final class Tax` means users cannot extend or implement Tax. That sounds strict, but it protects them: if we add a new field or method later, nobody's subclass breaks. This is a backward-compatibility tool.

> **In simple words**
> Immutable means 'printed on paper': you can photocopy it with changes (copyWith), but you never erase the original.

Design decisions you will see in the code

- **Numbers are `num`.** So both `quantity: 2` and `quantity: 2.5` work, and an `int` variable can be passed without `.toDouble()`. In Dart, `13 == 13.0` is true and their hashCodes match, so equality still works.
- **tax vs taxes.** Both Invoice and InvoiceItem accept `tax:` (one) or `taxes:` (many). Passing both throws an ArgumentError - in release builds too, not only in debug.
- **Item taxes have 3 meanings:** not set (null) = use the invoice taxes; a list = use these instead; `const []` = tax-exempt item.
- **Logo lives on Business.** The requirements listed a logo on both Invoice and Business. Two places for the same thing is confusing, so the seller's logo is `Business.logo`.
- **dueDate is optional** for 'due on receipt' invoices.
- **Currency defaults to USD** so the quick start stays short.
- **metadata vs details.** details are printed on the PDF (PO number); metadata is never printed (your database ID).
- **No QR field yet.** Adding an optional field later is not a breaking change, so we add it only when it actually works.

Step 3 - Sealed class for the logo

```
sealed class InvoiceLogo {
  const factory InvoiceLogo.memory(Uint8List bytes) = MemoryInvoiceLogo;
  const factory InvoiceLogo.asset(String assetPath) = AssetInvoiceLogo;
  const factory InvoiceLogo.network(String url)     = NetworkInvoiceLogo;
}
```

`sealed` means the compiler knows the full list of logo kinds. In Phase 4, when we load the logo with a `switch`, Dart will error if we forget one kind. Users only see `InvoiceLogo` - the three concrete classes are hidden with `export ... show InvoiceLogo`.

A memory logo is equal to another only if it is the same byte buffer. Comparing two 2 MB images byte by byte every time you compare invoices would be slow.

Step 4 - Exceptions

`InvoiceException` is the base class. It is deliberately **not** sealed: if it were, adding a new exception type in v1.2 would break users' exhaustive switch statements. `InvoiceValidationException` carries a list of `InvoiceValidationIssue(field, message)`, for example `items[1].quantity: must not be negative`. The validator that fills it comes in Phase 3, together with the calculator, because some rules (fixed discount bigger than the subtotal) need the totals.

Step 5 - The public door: one export file

```
// lib/flutter_invoice_generator.dart
export 'src/models/invoice.dart';
export 'src/models/invoice_logo.dart' show InvoiceLogo;
...
```

Internal helpers (`utils/equality.dart`, `utils/tax_merge.dart`) are **not** exported, so we can change them any time.

Step 6 - Tests (42 passing)

File	What it proves
test/models/currency_test.dart	8 required currencies, unique codes, JPY = 0 decimals, custom currency, equality, copyWith, bad decimals rejected
test/models/tax_discount_test.dart	decimal tax rates, 13 == 13.0, short and long Discount forms are equal, percentage != fixed
test/models/parties_test.dart	Business/Customer/Payment fields, payment without IBAN, logo equality rules
test/models/invoice_item_test.dart	quantities 1, 2, 2.5, 0.75; the 3 tax meanings; both-tax error; list copied and read-only
test/models/invoice_test.dart	quick-start invoice, defaults, multiple taxes (CGST+SGST), immutability, equality, metadata key order
test/exceptions/...	messages list every issue; all catchable as InvoiceException

Tests import only `package:flutter_invoice_generator/flutter_invoice_generator.dart`, exactly like a real user. That also proves the exports are correct.

```
flutter pub get
flutter analyze        ->  No issues found!
flutter test           ->  +42: All tests passed!
dart format lib test   ->  0 changed
```

Try it yourself: change a value in `test/models/invoice_test.dart` so a test fails, then run `flutter test`.

Next (Phase 3): InvoiceCalculator + InvoiceTotals + InvoiceValidator, the exact minor-unit money math, and many calculation tests (1 x $100, 10% discount, 13% tax, decimals, rounding, large values).

Chapter 3 - Phase 3: The calculation engine

This phase is all about money math. No PDF yet - if the numbers are wrong, a beautiful PDF is worthless. We added 4 files and 65 new tests (107 in total).

```
lib/src/calculation/
  money.dart             (internal) exact decimals, rounding, allocation
  totals_engine.dart     (internal) the math, in BigInt minor units
  invoice_totals.dart    (public)   InvoiceTotals, InvoiceLineTotals,
                                    InvoiceTaxLine
  invoice_calculator.dart(public)   InvoiceCalculator.calculate()
lib/src/validation/
  invoice_validator.dart (public)   InvoiceValidator.validate()
```

How to use it

```
final totals = InvoiceCalculator.calculate(invoice);
totals.subtotal;           // 700.0
totals.discountAmount;     // 70.0
totals.taxableAmount;      // 630.0
totals.taxLines;           // [InvoiceTaxLine(VAT: 81.9)]
totals.total;              // 711.9
totals.totalInMinorUnits;  // 71190  (cents - for Stripe etc.)
totals.lines[0].lineTotal; // 500.0
```

Idea 1 - Never do money math with double

A `double` stores numbers in binary. Most decimals (0.1, 1.005...) cannot be stored exactly, so tiny errors appear and then rounding goes the wrong way:

```
1.005 * 100           // 100.49999999999999  -> rounds to 1.00  (wrong!)
our engine: 1.005     // exactly 1005 / 1000 -> rounds to 1.01  (right)
```

How we fixed it: Dart prints every double in its shortest exact form (`1.005.toString() == '1.005'`). We read that text and turn it into two whole numbers: digits = 1005, scale = 3 (meaning 1005 / 10^3). This is `ExactDecimal` in money.dart. All math then uses `BigInt` (whole numbers of any size).

> **In simple words**
> We stop writing '1.005 dollars' and write '1005 thousandths of a dollar'. Whole numbers are always exact.

Idea 2 - Why BigInt and not int?

On Flutter Web, Dart's `int` is really a JavaScript number. It is only exact up to 2^53 (about 9 quadrillion). `quantity x price x 10^digits` can pass that for big invoices. BigInt has no limit and gives the same answer on Android, iOS, desktop and Web.

Idea 3 - Rounding: half away from zero

Exact value	Rounded (2 decimals)	Why
0.124	0.12	below half - down
0.125	0.13	exactly half - away from zero (up)
2.5 (JPY, 0 decimals)	3	not 2 - we do NOT use banker's rounding
-0.125	-0.13	away from zero, symmetric

The function is `roundDiv(numerator, denominator)`: divide, look at the remainder, and if remainder x 2 >= denominator, move one step away from zero.

Idea 4 - Round each value once, then only add

```
line   = round(quantity x unitPrice)  - round(item discount)
subtotal = sum(lines)                        <- exact, no rounding
discount = round(subtotal x %)  or fixed
taxable  = subtotal - discount               <- exact
tax      = round(taxable_for_that_rate x %)  <- once per tax rate
total    = taxable + sum(taxes)              <- exact
```

Because every 'sum' step is exact, the PDF can never show numbers that do not add up. We have a test that builds **300 random invoices** and checks this for every single one.

> **Why round tax per rate, not per line?**
> 3 lines of 0.10 at 5%: per line 0.005 rounds to 0.01, times 3 = 0.03. On the sum, 0.30 x 5% = 0.015 rounds to 0.02. Per-line rounding adds up rounding errors, so most accounting systems round tax on the total per rate. We do the same (there is a test for it).

Idea 5 - Sharing the discount fairly (largest remainder)

If items have **different** tax rates, we must know how much of the invoice discount belongs to each line. Example: discount of $1.00 over three $1 items. 100 cents / 3 = 33.33 each.

Step	Item A	Item B	Item C	Sum
Exact share	33.33	33.33	33.33	99.99?
Round down	33	33	33	99
1 cent left - goes to largest remainder (tie: first)	34	33	33	100 OK

This is `allocateProportionally()`. The shares always add up to exactly the discount. (In Chapter 1 I wrote 'leftover to last item'; largest remainder is fairer, so I changed it.)

Mixed taxes example (tested)

Item	Line	Share of 10% disc.	Taxable	Tax
A (uses invoice VAT 13%)	100	10	90	VAT 11.70
B (own GST 5%)	100	10	90	GST 4.50
C (taxes: const [] = exempt)	100	10	90	-

Item	Line	Share of 10% disc.	Taxable	Tax
Total	300	30	270	16.20 -> total 286.20

Idea 6 - The validator

`InvoiceValidator.validate(invoice)` returns a list of problems (empty = valid). `calculate()` calls it first and throws `InvoiceValidationException` if anything is wrong. It works in two rounds:

- **Round 1 - simple checks:** blank invoice number/names, no items, due date before issue date, negative or NaN quantity/price, tax not 0-100%, blank or duplicated tax, percentage discount not 0-100%, negative fixed discount.
- **Round 2 - needs the math** (only if round 1 passed, because the math would crash on NaN): fixed discount bigger than its line or the subtotal, and totals bigger than 10^15 minor units.

> **A bug the tests caught**
> An item with a $25 discount on a $20 line also made the subtotal negative, so the validator reported a second, confusing error on the invoice discount. Now it reports only the root cause. This is exactly why we write tests before building on top.

Why 10^15? Totals are returned as `double`. A double can hold any 15-digit number exactly. 10^15 cents is 10 trillion dollars - more than enough, and we can promise every value is exact.

Idea 7 - Public types you cannot construct

`InvoiceTotals` has its constructor marked `@internal`. Only our calculator should make one. That lets us add a new field in v1.1 without breaking anyone, because nobody else calls the constructor. Same idea for `abstract final class InvoiceCalculator`: it only has static methods, cannot be created, and cannot be extended.

Tests added in this phase

File	What it checks
test/calculation/money_test.dart	exact parsing (1.005, 1e-7, 1e21), half-away rounding, percent, conversion back, allocation
test/calculation/invoice_calculator_test.dart	1x$100, 2x$100, multiple items, zero qty, decimal qty/price, 0.1+0.2, 100 x 0.01, 10% / fixed / zero / 100% discount, item discounts, 13% tax, no tax, zero tax, CGST+SGST, mixed item taxes, per-rate rounding, 1.005, 0.125, JPY, 3-decimal currency, large values, 300 random invoices
test/validation/invoice_validator_test.dart	every rule above, plus 'reports every problem at once'

```
flutter analyze   ->  No issues found!
flutter test      ->  +107: All tests passed!

# Same tests on Web, using Brave (Chromium) instead of Chrome:
export CHROME_EXECUTABLE="/Applications/Brave Browser.app/Contents/MacOS/Brave Browser"
flutter test --platform chrome   ->  +107: All tests passed!
```

Next (Phase 4): add the `pdf` package, build `InvoiceGenerator.generate()` with the Classic template, fonts and logo loading; test 1-150 item invoices.